// INTERNET KE ANDAR · EP 02 · GUIDE

TCP vs UDP

@thestackexplained
FREE PDF GUIDE

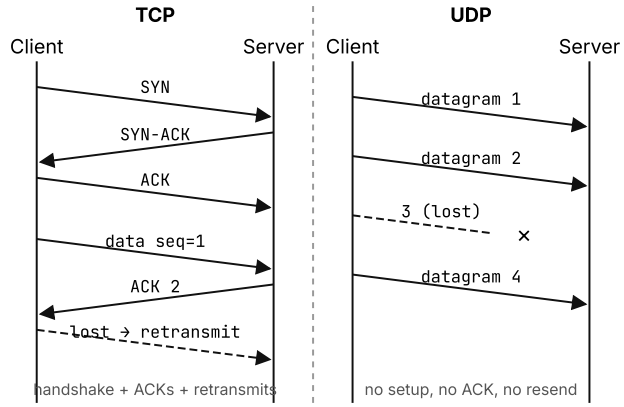
TL;DR · TCP = reliable, ordered byte stream (pay setup + retransmits). UDP = fast, independent datagrams (app handles loss). Pick by the cost of a lost packet.

01 Core concept

Both run on top of IP, which only does **best-effort** delivery: packets can be lost, duplicated or reordered. The transport layer decides what to do about that.

TCP (Transmission Control Protocol) is connection-oriented. It gives the app a **reliable, in-order byte stream** using sequence numbers, ACKs, retransmission, flow control and congestion control.

UDP (User Datagram Protocol) is connectionless. Each datagram is sent on its own, with only ports, length and a checksum. No handshake, no ACKs, no ordering. Simple bolein toh: TCP registered post hai, UDP live commentary.



Left: TCP sets up a connection and recovers lost data. Right: UDP just sends.

02 How it works, step by step

1. **TCP setup:** 3-way handshake SYN → SYN-ACK → ACK (≈1 RTT before data; TLS adds more on top).
2. **TCP transfer:** bytes are numbered; the receiver ACKs what it got; missing data is retransmitted after a timeout or 3 duplicate ACKs.
3. **Flow control:** the receiver advertises a window (how much it can buffer); the sender never exceeds it.
4. **Congestion control:** slow start + congestion avoidance (e.g. CUBIC, BBR) grow the send rate and cut it on loss/delay.
5. **TCP close:** FIN/ACK in each direction; the closing side waits in TIME_WAIT.
6. **UDP:** app calls send → one datagram → IP. Receiver gets it, or doesn't. Any reliability (seq numbers, retries, FEC) is the app's job.

03 Decision table

	TCP	UDP
Connection	Yes (handshake)	No
Delivery / order	Guaranteed, in order	Best effort, any order
Header	20–60 bytes	8 bytes
Unit	Byte stream (no message boundaries)	Datagram (boundaries kept)
Congestion control	Built in	None (app must be polite)
Use for	HTTP/1.1–2, APIs, payments, DB connections, SSH, file transfer	DNS queries, live video/voice, gaming, telemetry, QUIC/HTTP-3, WebRTC media

04 Implementation tips

- Default to **TCP** (or HTTP/gRPC on top) unless you have a real-time reason. Re-implementing reliability on UDP is hard.
- TCP is a stream: **frame your messages** (length prefix or delimiter). One `send()` ≠ one `recv()`.
- For UDP, keep datagrams **≤ ~1200 bytes** to avoid IP fragmentation; add sequence numbers if order matters; send newest state instead of resending old state (games, live video).
- Need reliability + no head-of-line blocking + fast setup? Use **QUIC** (HTTP/3) rather than rolling your own.
- Tune timeouts and keep connections alive (connection pooling) instead of opening a new TCP connection per request.

```
import socket
# TCP: connect, then a reliable stream
t = socket.create_connection(("example.com", 80))
t.sendall(b"GET / HTTP/1.1\r\nHost: example.com\r\n\r\n")
print(t.recv(4096)[:60])

# UDP: no connection, one datagram, may never arrive
u = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
u.settimeout(2)                      # app decides how long to wait / retry
u.sendto(b"ping", ("127.0.0.1", 9999))
```

05 Common pitfalls

- Saying “UDP is always faster”: per packet it has less overhead, but without congestion control it can make loss worse for everyone.
- Assuming TCP preserves message boundaries. It doesn't. Two sends can arrive as one recv, or one send as two.
- Head-of-line blocking: one lost TCP segment stalls all later data on that connection (a big reason HTTP/3 moved to QUIC).
- Forgetting that DNS falls back to TCP for large responses and zone transfers, so it's not “UDP only”.
- Firewalls/NAT often drop idle UDP mappings; long-lived UDP flows need keepalives.

06 Interview Q&A

Q. Why does TCP need a 3-way handshake, not 2?
Both sides must pick and confirm initial sequence numbers. The third ACK proves the client got the server's SYN, so old duplicate SYNs can't open bogus connections.

Q. How does TCP detect and recover a lost segment?
Missing ACKs. Retransmit on timeout (RTO), or fast retransmit after 3 duplicate ACKs, then congestion control reduces the window.

Q. Flow control vs congestion control?
Flow control protects the **receiver** (advertised window). Congestion control protects the **network** (congestion window). The sender uses the smaller of the two.

Q. Why is DNS mostly UDP?
A query and an answer usually fit in one datagram, so a handshake would double latency. The client simply retries on timeout; large responses switch to TCP.

Q. HTTP/3 runs on UDP. Isn't that unreliable?
QUIC adds reliability, ordering per stream, congestion control and TLS 1.3 in user space on top of UDP. It avoids TCP's cross-stream head-of-line blocking and can set up in 1 RTT (0-RTT on resume).

SCENARIO **Q. IPL final: a streaming app serves crores of live viewers, and the same users send UPI payments. Which transport for each, and why?**
Live video: UDP-based (WebRTC/QUIC or low-latency HLS over QUIC). A late frame is useless, so skip it and keep latency low. Payments: TCP or QUIC with TLS and an idempotency key. Every byte and the order matter, and a retry must not charge twice. Rejected answer: “UDP for everything because it's faster.”

// QUICK REVISION

- IP = best effort; TCP/UDP decide the rest
- TCP: handshake, seq + ACK, retransmit, in order
- Flow ctrl = receiver; congestion ctrl = network
- UDP: 8-byte header, no setup, no guarantees
- TCP is a stream: frame your messages
- Lost packet costly → TCP; late packet useless → UDP
- QUIC = reliability + TLS on UDP (HTTP/3)
- DNS: UDP first, TCP for big answers